

Agorithmique 2 : Sudoku

2024-2025

Un rapport au format PDF ainsi que le code doivent être envoyés avant le 19 février à 23:59 heure de Paris à l'adresse mail **frederic.hayek@hesias.fr**. Un oral de 30 minutes aura lieu le vendredi 21 février. Une présentation de 15 minutes de votre projet est attendue, suivie de 15 minutes de questions. Vous serez évalués sur le rendu (code et rapport) ainsi que sur l'oral, à 50% et 50%. Le rapport doit comprendre :

1. Les décisions de conception générales.
2. Les défis rencontrés et comment vous les avez surmontés.
3. Une évaluation de l'efficacité de votre solution.

1 Représentation de la Grille de Sudoku

Utilisez une structure de données appropriée pour représenter la grille Sudoku, par exemple une matrice 9×9 . Assurez-vous que votre représentation permet un accès et une modification efficaces des cellules.

2 Résolveurs Algorithmiques

Implémentez plusieurs méthodes pour résoudre les grilles de Sudoku :

1. Backtracking Récursif : Développez une solution de backtracking classique utilisant la récursion. Cette méthode essaie de remplir les cellules de manière séquentielle et recule lorsqu'une contradiction est détectée en réessayant avec de nouveaux chiffres dans les cellules précédentes jusqu'à trouver une solution ou épuiser toutes les possibilités.
2. Backtracking Itératif avec Pile : Créez une version itérative du backtracking en utilisant une pile pour gérer l'état de la grille. Chaque élément de la pile doit enregistrer l'état actuel de la grille et la position de la cellule en cours de résolution.
3. Techniques Heuristiques : Ajoutez des heuristiques pour optimiser l'algorithme de backtracking, comme l'heuristique de la "Minimum Remaining Value" (MRV).

4. Restructuration en Graphe : Modélisez la grille comme un graphe, où chaque cellule est un nœud, et les arêtes représentent les contraintes mutuelles (ligne, colonne, carré). Cela permet d'appliquer des algorithmes de propagation de contraintes issus de la théorie des graphes. Vous appliquerez l'algorithme de "coloration de graphe" donné.

3 Générateur de Grilles

Implémentez un générateur de Sudoku qui produit des puzzles aléatoires, tout en assurant l'unicité et la résolubilité de chaque grille.

Intégrez des algorithmes de génération qui varient la difficulté des puzzles, en fonction de critères tels que le nombre de cases initialement remplies et le niveau de technique requis pour la résolution.

4 Estimation de la Difficulté

Développez un système pour estimer la difficulté des grilles. Tenez compte des critères suivants :

1. Nombre de cellules initialement remplies.
2. Complexité des techniques de résolution nécessaires.
3. Analyse des possibilités pour chaque cellule.

5 Interface Utilisateur et Fonctionnalités de Jeu

Créez une interface simple en terminal (interface graphique optionnelle, mais peu notée) où les utilisateurs peuvent :

1. Entrer manuellement une grille de Sudoku.
2. Visualiser la solution de la grille étape par étape.
3. Générer des grilles de Sudoku par niveau de difficulté.
4. Jouer au Sudoku avec les options suivantes :
 - (a) Des indices sur demande et des vérifications automatiques des mouvements.
 - (b) Retour en arrière : le jeu se rappellera de l'état de la grille grâce à une liste chaînée de grilles.

6 Analyse de Performance

Mesurez et comparez les performances des différents algorithmes en fonction du temps d'exécution. Comparez les algorithmes itératif et récursif, avec et sans l'heuristique, et les comparer avec la résolution de la coloration de graphe associée.

Faites une grille de taille adaptable : Permettez des tailles de grille personnalisées, par exemple 4×4 ou 16×16 , ainsi que des tailles arbitraires $n \times n$ (carré parfait). Analysez la performance en fonction de la taille de l'entrée.